Reducibility 04

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Undecidability Proofs

- So far, we have only one undecidable language:

$$A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts } w\}$$

- This is a fact since we already prove it
- Given a language A, how to prove that it is undecidable?
- First, we are going to use the proof by contradiction
  - Assume that A is decidable
  - Show that since A is decidable, A_{TM} is decidable
  - Contradict with the fact that A_{TM} is undecidable
  - A is undecidable

Halting Problem

- From a programmer point of view:
  - Given a method `foo` and an input `x`, there is no algorithm that can check whether `foo` will halt on input `x`
    - For some methods, you may be able to spot that it will halt or will not halt but not all
    - But there are infinite number of methods and infinite number of inputs
- From the computation theory point of view:
  - Given a Turing machine M and input string w, there is no algorithm that can check whether M will halt on input w
  - Formally,

$$HALT_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM that halts on input } w\}$$

- To prove halting problem, we just need to show that $HALT_{\text{TM}}$ is undecidable

- First, assume that $HALT_{\text{TM}}$ is decidable
- Next, we need to show that since $HALT_{\text{TM}}$ is decidable A_{TM} is decidable
  - We need to take the advantage of $HALT_{\text{TM}}$ being decidable to help us decide A_{TM}
- What does it mean when $HALT_{\text{TM}}$ is decidable?
  - There exists a TM R that decides $HALT_{\text{TM}}$
- What TM R (a decider for $HALT_{\text{TM}}$) can do?
  - If we run R on input $\langle M, w \rangle$ where M is a TM and w is a string, R will either accept or reject $\langle M, w \rangle$
    - If R accepts $\langle M, w \rangle \rightsquigarrow \langle M, w \rangle \in HALT_{\text{TM}}$
      M **halts on input** w
    - If R rejects $\langle M, w \rangle \rightsquigarrow \langle M, w \rangle \notin HALT_{\text{TM}}$
      M **does not halts on input** w
  - Can R loops in definitely on input $\langle M, w \rangle$?
    - No because R is a **decider**

- When $HALT_{\text{TM}}$ is decidable, given a TM M' and a string w', we know whether M' halts on input w'
  - By running R on input $\langle M', w' \rangle$
    - If R accepts $\langle M', w' \rangle \rightsquigarrow \langle M', w' \rangle \in HALT_{\text{TM}}$
      M' **halts on input** w'
    - If R rejects $\langle M', w' \rangle \rightsquigarrow \langle M', w' \rangle \notin HALT_{\text{TM}}$
      M' **does not halt on input** w'
- We need to use TM R to help us decide A_{TM}
- But what does it mean when A_{TM} is undecidable?
  - Given a TM M and a string w, we have no idea whether
    - M accepts w or
    - M does not accepts w
  - If we can distinguish between M accepts w and M does not accept w, A_{TM} is decidable

- What if we can construct TM M' and w' (from the TM M and the string w) such that
  - If M accepts w, M' halts on input w'
  - If M does not accept w, M' does not halt on input w'

  and give $\langle M', w' \rangle$ to TM R?
- By running R on the above input $\langle M', w' \rangle$:
  - If R accepts $\langle M', w' \rangle$:
    - R accepts $\langle M', w' \rangle \rightsquigarrow \langle M', w' \rangle \in HALT_{\text{TM}}$
    - $\langle M', w' \rangle \in HALT_{\text{TM}} \rightsquigarrow M'$ halts on input w'
    - M' halts on input $w' \rightsquigarrow M$ accepts w
    - M accepts $w \rightsquigarrow \langle M, w \rangle \in A_{\text{TM}}$
  - If R rejects $\langle M', w' \rangle$:
    - R rejects $\langle M', w' \rangle \rightsquigarrow \langle M', w' \rangle \notin HALT_{\text{TM}}$
    - $\langle M', w' \rangle \notin HALT_{\text{TM}} \rightsquigarrow M'$ does not halt on input w'
    - M' does not halt on input $w' \rightsquigarrow M$ does not accept w
    - M does not accept $w \rightsquigarrow \langle M, w \rangle \notin A_{\text{TM}}$
- This gives us the conclusion that A_{TM} is decidable

- Again, we need to construct TM M' and w' (from the TM M and the string w) such that
  - If M accepts w, M' halts on input w'
  - If M does not accept w, M' does not halt on input w'
- Here is an example of TM M':

  $M' =$ "On input x:
  1. Run M on input w
  2. If M accepts w, $accept$. If M rejects w, enter infinite loop."
- Note that there are infinite number of TMs that satisfy the above specification
- Let's analyze the behavior of the above TM M' case-by-case

- The TM M':

  $M' =$ "On input x:

  1. Run M on input w
  2. If M accepts w, *accept*. If M rejects w, enter infinite loop."

- What if M accepts w ($\langle M, w \rangle \in A_{\text{TM}}$)?
  - M accepts $w \rightsquigarrow M'$ accepts all input strings
  - M' accepts all input strings $\rightsquigarrow M'$ halts on all input strings
  - M' halts on all input strings $\rightsquigarrow M'$ halts on ε
  - M' halts on $\varepsilon \rightsquigarrow \langle M', \varepsilon \rangle \in HALT_{\text{TM}}$

- The TM M':

  $M' =$ "On input x:

  1. Run M on input w
  2. If M accepts w, $accept$. If M rejects w, enter infinite loop."

- What if M rejects w ($\langle M, w \rangle \notin A_{\text{TM}}$)?
  - M rejects $w \rightsquigarrow M'$ enters infinite loop on all input strings
  - M' enters infinite loop on all input strings $\rightsquigarrow M'$ does not halt on all input strings
  - M' does not halt on all input strings $\rightsquigarrow M'$ does not halt on ε
  - M' does not halt on $\varepsilon \rightsquigarrow \langle M', \varepsilon \rangle \notin HALT_{\text{TM}}$

- The TM M':

  $M' =$ "On input x:

  1. Run M on input w
  2. If M accepts w, $accept$. If M rejects w, enter infinite loop."

- What if M loops indefinitely on w ($\langle M, w \rangle \notin A_{\text{TM}}$)?
  - M loops indefinitely on $w \rightsquigarrow M'$ loops indefinitely (in step 1) on all input strings
  - M' loops indefinitely on all input strings $\rightsquigarrow M'$ does not halt on all input strings
  - M' does not halt on all input strings $\rightsquigarrow M'$ does not halt on ε
  - M' does not halt on $\varepsilon \rightsquigarrow \langle M', \varepsilon \rangle \notin HALT_{\text{TM}}$

- Again, we just construct TM M' and $w' = \varepsilon$ from a TM M and a string w such that
  - If $\langle M, w \rangle \in A_{\text{TM}}$, $\langle M', \varepsilon \rangle \in HALT_{\text{TM}}$
  - If $\langle M, w \rangle \notin A_{\text{TM}}$, $\langle M', \varepsilon \rangle \notin HALT_{\text{TM}}$
- Now, we can simply run TM R (a decider for $HALT_{\text{TM}}$ to check whether $\langle M', \varepsilon \rangle$ is in $HALT_{\text{TM}}$
  - If R accepts $\langle M', \varepsilon \rangle \rightsquigarrow \langle M', \varepsilon \rangle \in HALT_{\text{TM}} \rightsquigarrow \langle M, w \rangle \in A_{\text{TM}}$
  - If R rejects $\langle M', \varepsilon \rangle \rightsquigarrow \langle M', \varepsilon \rangle \notin HALT_{\text{TM}} \rightsquigarrow \langle M, w \rangle \notin A_{\text{TM}}$
- Let's see the whole proof where we construct a TM S that decides A_{TM} where
  - S constructs TM M' as we discussed earlier and
  - uses R as a helper machine to help decide A_{TM}

$HALT_{\text{TM}}$

Prove that $HALT_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM that halts in input } w\}$ is undecidable.

Proof: Assume that $HALT_{\text{TM}}$ is decidable. Since $HALT_{\text{TM}}$ is decidable, there exists a TM R that decides $HALT_{\text{TM}}$. Let construct a TM S (to decide A_{TM}) as follows:
$S =$ "On input $\langle M, w\rangle$ where M is a TM and w is a string:

1. Construct TM M' as follows:

   $M' =$ "On input x:
   1. Run M on input w
   2. If M accepts w, *accept*. If M rejects w, enter infinite loop."
2. Run R on input $\langle M', \varepsilon\rangle$
3. If R accepts $\langle M', \varepsilon\rangle$, *accept*. If R rejects $\langle M', \varepsilon\rangle$, *reject*."

Next, we need to prove that TM S is a decider for A_{TM}.

- Prove that if $\langle M, w\rangle \in A_{\text{TM}}$, S accepts $\langle M, w\rangle$: Assume that $\langle M, w\rangle \in A_{\text{TM}}$. Since $\langle M, w\rangle \in A_{\text{TM}}$, M is a TM that accepts w. Since M accepts w, from the definition of TM M', M' halts on all strings including ε. Since M' halts on ε, $\langle M', \varepsilon\rangle \in HALT_{\text{TM}}$. Since R is a decider for $HALT_{\text{TM}}$, by running R on input $\langle M', \varepsilon\rangle$, R will accept $\langle M', \varepsilon\rangle$. Since R accepts $\langle M', \varepsilon\rangle$, S accepts $\langle M, w\rangle$.
- Prove that if $\langle M, w\rangle \notin A_{\text{TM}}$, S rejects $\langle M, w\rangle$: Assume that $\langle M, w\rangle \notin A_{\text{TM}}$. Since $\langle M, w\rangle \notin A_{\text{TM}}$, M is a TM that does not accept w. Since M does not accept w, from the definition of TM M', M' does not halt on all strings including ε. Since M' does not halt on ε, $\langle M', \varepsilon\rangle \notin HALT_{\text{TM}}$. Since R is a decider for $HALT_{\text{TM}}$, by running R on input $\langle M', \varepsilon\rangle$, R will reject $\langle M', \varepsilon\rangle$. Since R rejects $\langle M', \varepsilon\rangle$, S rejects $\langle M, w\rangle$.

This show that TM S is a decider for A_{TM}. Therefore, A_{TM} is decidable $\rightsquigarrow$ contradiction. Therefore, $HALT_{\text{TM}}$ is undecidable.

Undecidability

- Let's recap and make some notes
- To prove that a language A is undecidable:
  - Assume that A is decidable
  - Since A is decidable, there exists a TM R that decides A
  - Next, we try to decide A_{TM} using R as a helper machine
  - To decide A_{TM}, we need to know whether $\langle M, w \rangle \in A_{\mathrm{TM}}$
    - In other words, we need to know whether M accepts w
  - Since R is a decider for the language A, R can tell you whether a string s is in A
- So, we need to construct a string s such that whether s is in A depending on whether M accepts w

Undecidability

- To construct a string s such that whether s is in A depending on whether M accepts w, there are two choices:
  1. Choice #1:
     - If M accepts w, $s \in A$
     - If M does not accept w, $s \notin A$
  2. Choice #2:
     - If M accepts w, $s \notin A$
     - If M does not accept w, $s \in A$
- Which one?
  - It depends on the problem
  - For some problems, either choices will work
  - For some problems, only one of those choices will work

Undecidability

- From the previous example, we have two choices as well:
  1. Choice #1:
     - If M accepts w, $\langle M', w' \rangle \in HALT_{\text{TM}}$
     - If M does not accept w, $\langle M', w' \rangle \notin HALT_{\text{TM}}$
  2. Choice #2:
     - If M accepts w, $\langle M', w' \rangle \notin HALT_{\text{TM}}$
     - If M does not accept w, $\langle M', w' \rangle \in HALT_{\text{TM}}$
- We picked the first choice
- If you try, you will see that you cannot construct a TM M' that satisfies the second choice
  - The hard part is when M loops indefinitely on input w

- **Problem**: Given a TM M, how to check whether $L(M) = \emptyset$?
  - Unfortunately, we cannot
  - Note that a TM may have a path from the start state to its accept state but it accepts no string
- Formally:

$$E_{\mathrm{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$$

- To show that there is no algorithm that can solve the above problem, we need to show that E_{TM} is undecidable

- Again, for the sake of contradiction, assume that E_{TM} is decidable
  - We need to show that if E_{TM} is decidable, A_{TM} is decidable
- Since E_{TM} is decidable, there exists a TM R that decides E_{TM}
- Again, in plain English, given a TM M', we know whether $L(M') = \emptyset$ by simply run R on input $\langle M' \rangle$
  - If R accepts $\langle M' \rangle \rightsquigarrow \langle M' \rangle \in E_{\mathrm{TM}} \rightsquigarrow L(M') = \emptyset$
  - If R rejects $\langle M' \rangle \rightsquigarrow \langle M' \rangle \notin E_{\mathrm{TM}} \rightsquigarrow L(M') \neq \emptyset$
- We are going to use the same method as in previous example:
  - Since we are trying to decide A_{TM}, we need to know whether M accepts w
  - We are going to construct a TM M' from a given TM M and a string w such that $L(M')$ (whether it is empty or not) depends on whether M accepts w

- **Specification**: Construct a TM M' from a given TM M and a string w such that $L(M')$ (whether it is empty or not) depends on whether M accepts w
- According to the above specification, we have two choices:
  1. Choice #1:
     - If M accepts w, $L(M') = \emptyset$
     - If M does not accept w, $L(M') \neq \emptyset$
  2. Choice #2:
     - If M accepts w, $L(M') \neq \emptyset$
     - If M does not accept w, $L(M') = \emptyset$
- In this case, we will not be able to construct TM M' that satisfies the first choice

- Here is an example of TM M' (constructed from a TM M and a string w) that satisfies:
  - If M accepts w, $L(M') \neq \emptyset$
  - If M does not accept w, $L(M') = \emptyset$
- TM M':

  $M' =$ "On input x:
  1. Run M on input w
  2. If M accepts w, *accept*. If M rejects w, *reject*."
- From the above TM M':
  - If M accepts w, M' accepts all strings ($L(M') \neq \emptyset$)
  - If M does not accepts w:
    - If M rejects w, M' rejects all strings ($L(M') = \emptyset$)
    - If M loops indefinitely on input w, M' loops indefinitely on all strings ($L(M') = \emptyset$)
- Now, let's see the whole proof

E_{TM}

Prove that $E_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$ is undecidable.

Proof: Assume that E_{TM} is decidable. Since E_{TM} is decidable, there exists a TM R that decides E_{TM}. Let construct a TM S (to decide A_{TM}) as follows:
$S =$ "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct TM M' as follows:

   $M' =$ "On input x:
   1. Run M on input w
   2. If M accepts w, *accept*. If M rejects w, *reject*."
2. Run R on input $\langle M' \rangle$
3. If R accepts $\langle M' \rangle$, *reject*. If R rejects $\langle M' \rangle$, *accept*."

Next, we need to prove that TM S is a decider for A_{TM}.

- Prove that if $\langle M, w \rangle \in A_{\text{TM}}$, S accepts $\langle M, w \rangle$: Assume that $\langle M, w \rangle \in A_{\text{TM}}$. Since $\langle M, w \rangle \in A_{\text{TM}}$, M is a TM that accepts w. Since M accepts w, from the definition of TM M', M' accepts all strings. In other words, $L(M') \neq \emptyset$. Since $L(M') \neq \emptyset$, $\langle M' \rangle \notin E_{\text{TM}}$. Since R is a decider for E_{TM}, by running R on input $\langle M' \rangle$, R will reject $\langle M' \rangle$. Since R rejects $\langle M' \rangle$, S accepts $\langle M, w \rangle$.
- Prove that if $\langle M, w \rangle \notin A_{\text{TM}}$, S rejects $\langle M, w \rangle$: Assume that $\langle M, w \rangle \notin A_{\text{TM}}$. Since $\langle M, w \rangle \notin A_{\text{TM}}$, M is a TM that does not accept w. Since M does not accept w, from the definition of TM M', M' accepts no strings. In other words, $L(M') = \emptyset$. Since $L(M') = \emptyset$, $\langle M' \rangle \in E_{\text{TM}}$. Since R is a decider for E_{TM}, by running R on input $\langle M' \rangle$, R will accept $\langle M' \rangle$. Since R accepts $\langle M' \rangle$, S rejects $\langle M, w \rangle$.

This show that TM S is a decider for A_{TM}. Therefore, A_{TM} is decidable $\rightsquigarrow$ contradiction. Therefore, E_{TM} is undecidable.